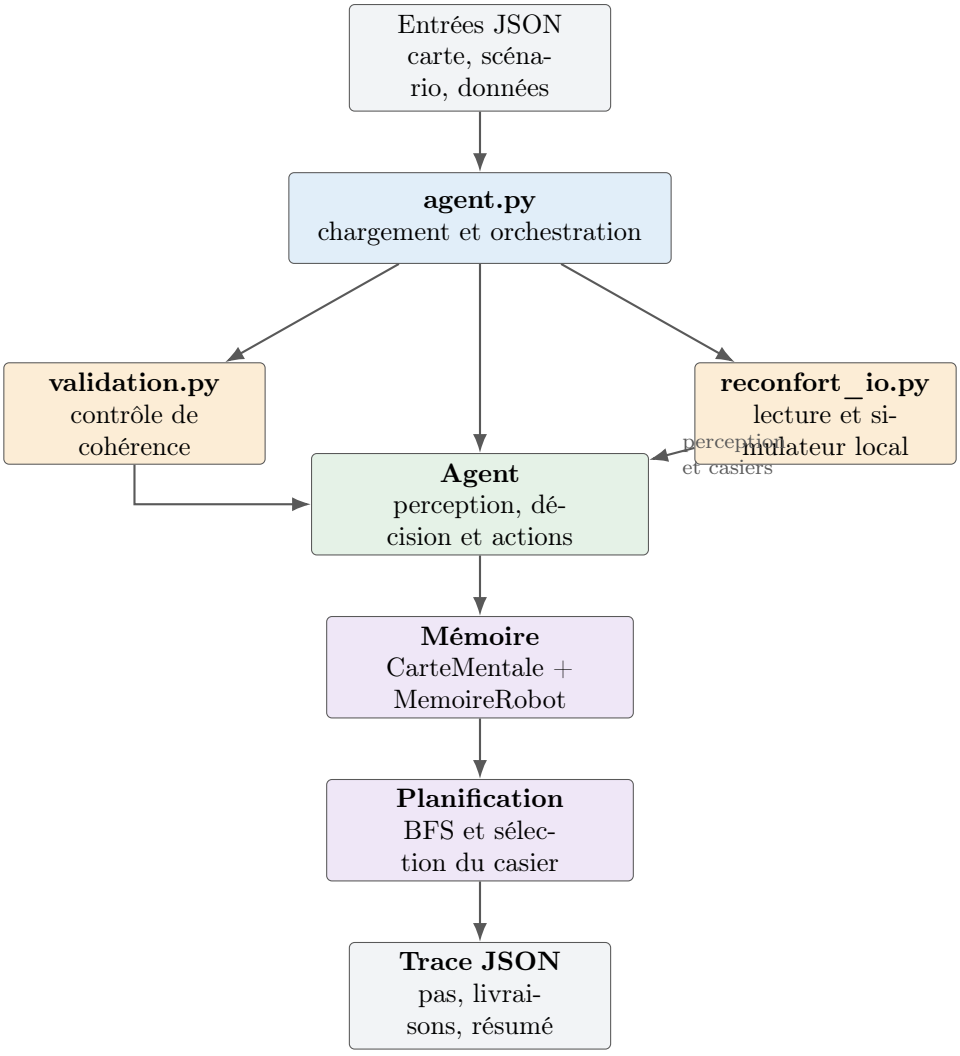
Rapport d'analyse

Robot de réconfort

BOUBENIA Walid BARRE-GUETROT Ilan

1 Architecture

L'architecture du projet est organisée autour d'un contrôleur principal et de composants spécialisés :



Les flèches représentent le flux de contrôle et d'informations.

Les frontières entre les composants sont volontaires :

— **Perception :** `PerceptionLocale`, dans `reconfort_io.py`, est la seule interface qui consulte la grille réelle. Elle expose uniquement les quatre cases voisines et le contenu du casier lorsque le robot est devant l'armoire. Cela respecte la contrainte de perception locale.

— **Mémoire :** `CarteMentale` et `MemoireRobot`, dans `agent.py`, stockent ce que le robot a découvert et son état courant. Cette séparation empêche la planification de lire directement la carte réelle et permet de tester la mémoire indépendamment du simulateur.

— **Décision :** `Agent.choisir_action` utilise la perception et la mémoire pour choisir `AVANCER`, `CONSULTER`, `CHERCHER`, `PRENDRE`, `DONNER` ou `ATTENDRE`. La décision ne modifie pas directement la grille simulée.

— **Action :** `executer_action` applique l'action à l'état interne, appelle l'interface de prise si nécessaire et journalise l'état avant l'action. `Trace` est séparée afin que la journalisation n'influence pas la décision.

— **Validation :** `validation.py` est exécuté avant la simulation. Il vérifie la cohérence des cartes, scénarios, dictionnaires et casiers, au lieu de disperser ces contrôles dans l'agent.

Le flux principal est donc :

`charger_entrees` $\longrightarrow$ `valider_entrees` $\longrightarrow$ création du simulateur et de l'agent
$\longrightarrow$ `percevoir` $\longrightarrow$ mémoriser $\longrightarrow$ `choisir_action`
$\longrightarrow$ `executer_action` $\longrightarrow$ `Trace`.

2 Stratégie de replanification

Le robot **replanifie entièrement** son chemin, plutôt que de réparer une partie du plan. À chaque itération de `traiter_demande`, il observe les quatre voisins, met à jour la carte mentale, puis appelle indirectement `plus_court_chemin`, qui effectue un BFS depuis la position courante vers une case adjacente à la cible.

Lorsqu'un mur est découvert, `memoriser_perception` inscrit la nature `mur` dans `CarteMentale`. Le BFS suivant exclut alors cette case et reconstruit un nouveau chemin avec les informations disponibles. Les cases encore inconnues sont provisoirement considérées comme traversables ; elles seront corrigées par les perceptions suivantes.

Le coût d'une replanification est $\mathcal{O}(H \times L)$ dans le pire cas pour une carte de $H \times L$ cases, avec $\mathcal{O}(H \times L)$ de mémoire pour les parents et les cases visitées. Cette solution est plus simple et plus sûre qu'une réparation locale : elle garantit que le chemin utilisé tient compte de tous les murs connus. Son inconvénient est de recalculer le chemin après chaque action, même si la modification de la carte est faible. Pour les petites grilles du projet, ce coût reste acceptable.

3 Critique de la règle de repli

La règle imposée en section 6.2 est déterministe, mais elle est discutable : une émotion demandée est remplacée par une émotion voisine sur la roue, sans tenir compte de la signification de l'objet. Une personne triste peut donc recevoir un objet associé au dégoût. La proximité sur la roue de Plutchik ne garantit pas une proximité de besoin ou de confort. La règle est défendable comme solution de secours lorsque le casier exact est vide, mais elle ne devrait pas être interprétée comme une équivalence psychologique.

Une meilleure règle serait :

1. chercher d'abord un casier de l'émotion exacte et de l'intensité exacte ;
2. si aucun objet n'est disponible, chercher l'émotion exacte avec une intensité voisine ;
3. ensuite seulement, utiliser une émotion voisine, en privilégiant une table de compatibilité sémantique définie pour le projet ;
4. si aucun objet compatible n'existe, signaler explicitement l'échec plutôt que choisir automatiquement une émotion potentiellement inadaptée.

Dans l'implémentation actuelle, la distance émotionnelle est limitée à **2 positions** sur la roue. Ainsi, une distance de 0 correspond à l'émotion exacte, une distance de 1 à une émotion voisine et une distance de 2 à une émotion située deux positions plus loin. Les émotions à distance 3 ou 4 sont ignorées. La distance est circulaire : la première et la dernière émotion de la liste sont donc voisines. Cette limite est appliquée dans `Agent._casiers_candidats` par la condition `if distance > 2`.

Il n'existe pas de limite stricte équivalente pour l'intensité : les trois niveaux restent candidats, mais ils sont classés selon leur écart avec l'intensité demandée. Le tri privilégie donc d'abord l'émotion exacte, puis l'intensité la plus proche.

Pour mettre cette règle en œuvre, il faudrait modifier `Agent._casiers_candidats` dans `agent.py` afin de produire des priorités par niveaux de compatibilité, au lieu d'utiliser uniquement la distance circulaire.

4 Trois situations où le robot se comporte mal

4.1 Échec explicite : une émotion inconnue

Si une demande ne contient aucun mot du dictionnaire, `_lire_emotion_message` renvoie `None` et `emotion_indeterminee` est placé à `True`. La demande est alors classée en `ATTENDRE` et enregistrée comme un échec avec le motif `emotion indeterminee`. Le robot ne peut donc pas choisir d'objet ni effectuer la livraison. Ce comportement est volontaire : sans émotion identifiée, une réponse automatique serait arbitraire.

Un ancien problème d'état mémoire a été corrigé : le drapeau `emotion_indeterminee` est maintenant remis à `False` au début de `traiter_demande`. Une demande inconnue ne bloque donc plus les demandes valides suivantes.

4.2 Échec explicite : une armoire vide

Après identification de l'émotion, le robot explore les casiers candidats. Lorsque tous les casiers compatibles ont été observés comme vides, `choisir_action` renvoie `ATTENDRE`. La livraison est alors enregistrée avec le motif `armoire vide`.

Ce cas est correctement signalé, mais aucune solution de remplacement n'est proposée lorsque tous les objets pertinents sont indisponibles. Une évolution possible serait de proposer un objet d'une autre catégorie selon une règle de compatibilité explicite, ou de transmettre la demande à un autre robot.

4.3 Échec explicite : un résident inaccessible

Si aucun chemin vers une case adjacente au résident n'est trouvé dans la carte mentale, `_direction_vers` renvoie `None`. Le robot s'arrête avec l'action `ATTENDRE` et la trace indique `resident inaccessible`. Il ne tente pas de déplacer un mur et ne produit pas de fausse livraison.

La limite est que `ATTENDRE` est utilisé comme état terminal : le robot ne dispose pas d'une stratégie de recherche plus avancée ou de coopération pour rendre la cible accessible. Pour améliorer ce comportement, il faudrait ajouter une phase d'exploration contrôlée, ou un motif d'échec distinct indiquant que la zone a été entièrement explorée.

Autre limite étudiée. La règle de repli par distance circulaire peut aussi fournir un objet d'une émotion voisine peu adaptée, par exemple un objet associé au dégoût pour une personne triste. Ce point n'est pas un bug par rapport à l'énoncé : le code applique bien la règle imposée. Il est analysé dans la section précédente comme une limite de conception.